kasan 原理和应用实例

1. kasan 简介

kasan 全称是 kernel address sanitizer，用于检测内存访问越界，释放之后使用等内存非法操作。每 8 字节的内存用 1 字节的 shadow memory 标记内存使用状态，每次读/写内存的时候都会检测本次访问是否合法。如果发现有非法访问，可以捕获内存非法访问的现场，打印出堆栈信息和内存信息等供开发者进行问题分析。

2. shadow memory 填充

在打开 kasan 的内核中，系统初始化阶段会预留 shadow memory 区域，每 8 个字节的内存对应 1 个字节的 shadow memory，也就意味着打开 kasan 之后系统至少有八分之一的额外内存开销。每次申请内存之后，会填充对应的 shadow memory。如果申请的是 8 字节，对应的 shadow memory 填充为 0，如果申请是 n 字节（1<n<=7），对应的 shadow memory 填充为 n，如果申请的是 N 字节（n>8），会把 N/8 个 shadow memory 填充为 0，最后一个 shadow memory 填充为 N%8。还有一些特殊的小于零的值用来标记内存使用的不同状态，比如页面/slab 已经释放，新的 slab 内存，redzone 区域等。图 1 是特殊的标记。

```
#define KASAN_FREE_PAGE         0xFF  /* page was freed */
#define KASAN_PAGE_REDZONE      0xFE  /* redzone for kmalloc_large allocations */
#define KASAN_KMALLOC_REDZONE   0xFC  /* redzone inside slub object */
#define KASAN_KMALLOC_FREE      0xFB  /* object was freed (kmem_cache_free/kfree) */
#define KASAN_GLOBAL_REDZONE    0xFA  /* redzone for global variable */
```

图 1 kasan 特殊的 shadow memory 标记

图 2 以 slab 内存为例说明 shadow memory 填充情况，为了简化说明，去掉了 redzone 内存。当 slab 分配器从伙伴系统中申请新的 slab 时，此时 slab 链表是空闲的，会把全部 shadow memory 设置为 KASAN_KMALLOC_REDZONE（0xFC）。当分别从 slab 链表中申请 8 字节，6 字节，11 字节内存时，shadow memory 会变为 0x0,0x6,0x0,0x3。然后释放 6 字节内存的时候，shadow memory 变为 0x0,0xFB,0x0,0x3。

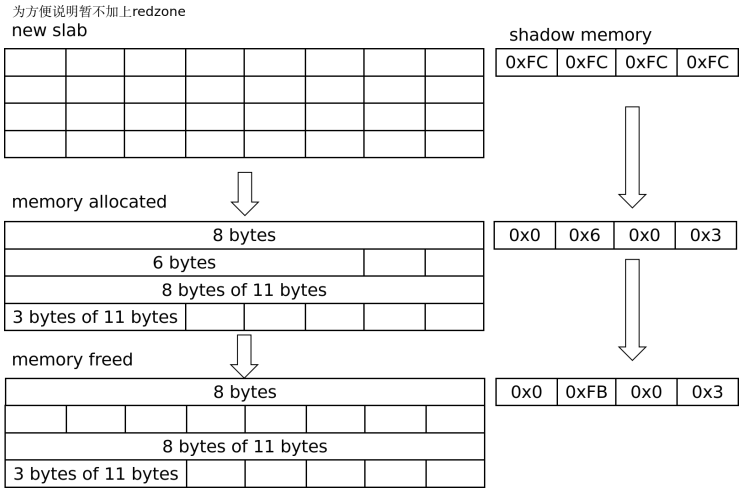

图 2 slab 内存申请释放过程 shadow memory 值

3. shadow memory 检测

打开 kasan 之后，编译器会在每次内存操作指令之前插入一条跳转到 __asan_loadX/__asan_stroreX 函数（X 是访问内存的长度 1/2/4/8/16，load 是在读内

存指令之前，store 是加在写内存指令之前）的指令，__asan_loadX/__asan_stroreX 函数是用来对 shadow memory 做检测的。反汇编 vmlinux 就可以看到很多如下图所示的指令：

```
0xffff80000065beb0 <+144>: asr  x21, x21, #6
0xffff80000065beb4 <+148>: add x21, x1, x21, lsl #16
0xffff80000065beb8 <+152>: bl0xffff8000002fe640 <__asan_load4>
0xffff80000065bebc <+156>: ldr  w23, [x20, #12]
0xffff80000065bec0 <+160>: add x19, x21, x22
0xffff80000065bec4 <+164>: cbz  w23, 0xffff80000065bf04 <mpi_read_raw_from_sgl+228>
0xffff80000065bec8 <+168>: mov   x0, x19
0xffff80000065becc <+172>: bl0xffff8000002fe4a0 <__asan_load1>
0xffff80000065bed0 <+176>: ldrbw0, [x21, x22]
0xffff80000065bed4 <+180>: cbnz  w0, 0xffff80000065bf78 <mpi_read_raw_from_sgl+344>
0xffff80000065bed8 <+184>: mov   w21, #0x1                // #1
0xffff80000065bedc <+188>: b  0xffff80000065bef0 <mpi_read_raw_from_sgl+208>
0xffff80000065bee0 <+192>: bl0xffff8000002fe4a0 <__asan_load1>
0xffff80000065bee4 <+196>: add w21, w21, #0x1
0xffff80000065bee8 <+200>: ldrbw0, [x19]
```

图 3 插入的__asan_loadX 指令

以__asan_load1 为例，如果访问的地址对应的 shadow memory 值为 0，表明访问有效。如果访问的地址对应的 shadow memory 值不为 0，表明该地址所在的 8 个字节不是全部为有效内存，需要做下一步做判断。取地址的低 3 位，如果该值大于等于 shadow memory 的值，表明本次访问是非法访问。会在 kasan report 中报出 bug，根据 shadow memory 的值来初步判断是越界访问还是释放之后使用等非法访问。很显然在上面的流程中有个问题，以上图 2 为例，如果访问申请出来的 6 字节中的某个地址加上一个偏移之后的地址，由于程序错误，导致偏移值计算错误，访问到后面的申请出来的 11 字节的有效地址范围中，而且恰好在前八个字节范围内，这时候是检测不到非法访问的。也就是有概率检测不到越界访问。为了解决这一问题，kasan 引入了 redzone 机制，在 shadoe memory 中加上额外的 redzone 区域，并设置为 0xFC，一旦踩中这个区域，也会报出 bug。从而大大降低漏报的概率。

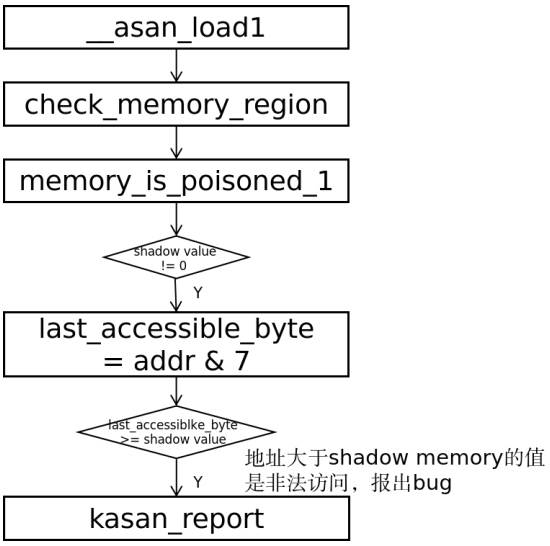

图 4 shadow memory 的检测

4. 应用实例

问题现象：x86 项目 4.4 内核，启动阶段会概率性报空指针，关机时候也会概率性报空指针。有时候即使启动正常，客户加载第三方驱动也会概率性失败，同样报的空指针。

问题初步分析：从调用栈来看，空指针来自访问 proc 文件系统，首先想到的是打印出文件名。加上打印信息之后测试发现每次出错访问的文件名都不一样，具有随机性可以排除是某个驱动创建的 proc 目录有问题。更倾向于是内存踩踏导致的问题。于是打开 KASAN 复现问题，发现报出两处错误，一个是 mpi 驱动中有访问越界，一个是 intel 的声卡驱动

中有 use-after-free。

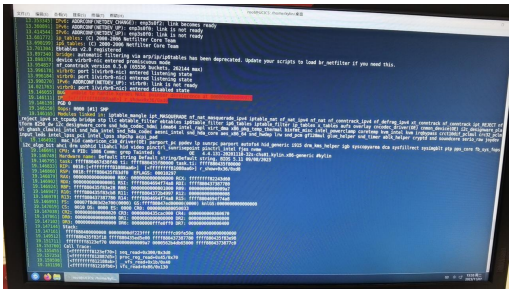

图 5 空指针报错

4.1 mpi 驱动内存越界

[36.618606] ==========

[36.628174] BUG: KASAN: slab-out-of-bounds in mpi_read_raw_from_sgl+0x342/0x400 at addr 0000000044d171e0

[36.637259] Read of size 1 by task modprobe/1229

[36.646990] ==========

[36.655453] BUG kmalloc-512 (Not tainted): kasan: bad access detected

[36.663969] ----------

.....

[36.785590] CPU: 1 PID: 1229 Comm: modprobe Tainted: G B 4.4.131-20231109.kylin.x86+ #1

[36.785593] Hardware name: Default string Default string/Default string, BIOS 5.11 09/08/2023

[36.785599] ffff8803e68c0000 1690832434a6f7ca ffff8803e2267560 ffffffff8163f7c8

[36.785604] ffff8803ee404c80 ffff8803e68c1068 ffff8803e2267590 ffffffff81308e39

[36.785609] ffff8803ee404c80 ffffea000f9a3000 ffff8803e68c1068 0000000000000000

[36.785609] Call Trace:

[36.785616] [<00000000ea7dbc05>] dump_stack+0x63/0x8b

[36.785622] [<00000000bc0c89a3>] print_trailer+0xf9/0x150

[36.785627] [<0000000069b8d0ae>] object_err+0x34/0x40

[36.785633] [<00000000ddc0cae9>] kasan_report.part.2+0x232/0x530

[36.785637] [<0000000020b555dc>] ? kasan_kmalloc+0x5e/0x70

[36.785653] [<0000000015957d3f>] ? mpi_read_raw_from_sgl+0x342/0x400

[36.785657] [<000000004e2879a7>] ? __kmalloc+0xfb/0x260

[36.785661] [<00000000ecc4854c>] ? mpi_alloc_limb_space+0x19/0x20

[36.785667] [<00000000cc13fdbe>] kasan_report+0x25/0x30

[36.785671] [<0000000081ddf6d5>] __asan_load1+0x47/0x50

[36.785678] [<0000000015957d3f>] mpi_read_raw_from_sgl+0x342/0x400

[36.785684] [<000000002f15e25c>] rsa_verify+0x176/0x210

[36.785690] [<000000001356759e>] ? _rsa_sign.isra.4+0x80/0x80

[36.785695] [<000000001a92abe8>] ? memset+0x28/0x30

[36.785699] [<000000009f166a4b>] ? sg_init_table+0x27/0x40

[36.785705] [<000000008f70409c>] rsa_verify_signature+0x37a/0x5b0

[36.785711] [<00000000a41e8786>] ? public_key_verify_done+0x50/0x50

[36.785717] [<00000000563a82cc>] ? x509_request_asymmetric_key+0xfb/0x260

[36.785723] [<00000000563a82cc>] ? x509_request_asymmetric_key+0xfb/0x260

[36.785729] [<00000000c5c9f127>] ? key_default_cmp+0x50/0x50

[36.785735] [<00000000a712d515>] public_key_verify_signature+0x74/0x90

[36.785741] [<00000000b17c6866>] public_key_verify_signature_2+0x2d/0x40

```
[    36.785746]   [<00000000040b2e36>] verify_signature+0x77/0xa0
[    36.785752]   [<000000004d0a8f91>] pkcs7_validate_trust+0x378/0x3f0
[    36.785758]   [<00000000ab45f023>] system_verify_data+0xc6/0x150
[    36.785762]   [<0000000087bf14b6>] ? top_trace_array+0x75/0x75
[    36.785768]   [<00000000f53b1d3b>] ? filemap_write_and_wait_range+0x40/0x80
[    36.785774]   [<00000000cbad11f7>] ? dump_trace+0x123/0x310
[    36.785780]   [<000000006e28f797>] ? fsnotify+0x155/0x8d0
[    36.785786]   [<000000008beaa765>] mod_verify_sig+0x110/0x180
[    36.785793]   [<00000000c5000ce1>] ? enforce_signed_modules+0x20/0x20
[    36.785799]   [<000000009f073e07>] load_module+0x313/0x4980
[    36.785804]   [<0000000051eac302>] ? __fsnotify_parent+0x8f/0x240
[    36.785811]   [<00000000cd5ddad3>] ? __fsnotify_inode_delete+0x20/0x20
[    36.785817]   [<00000000415a3031>] ? native_set_pte_at+0x27/0x40
[    36.785822]   [<0000000051eac302>] ? __fsnotify_parent+0x8f/0x240
[    36.785830]   [<000000005ef2da40>] ? __fsnotify_update_child_dentry_flags.part.1+0x160/0x160
[    36.785837]   [<0000000031e98012>] ? module_frob_arch_sections+0x20/0x20
[    36.785842]   [<000000002a3298bd>] ? kernel_read+0xdb/0x120
[    36.785846]   [<00000000279fcb6a>] ? open_exec+0x50/0x50
[    36.785853]   [<000000003b2322bd>] ? copy_module_from_fd.isra.55+0x257/0x270
[    36.785860]   [<00000000d9ed234c>] ? resolve_symbol.isra.52+0x200/0x200
[    36.785866]   [<00000000aa733dcf>] ? security_capable+0x33/0x80
[    36.785872]   [<00000000cce9728d>] SYSC_finit_module+0x126/0x160
[    36.785878]   [<00000000b5d16112>] ? SYSC_init_module+0x210/0x210
[    36.785885]   [<00000000500a7eee>] SyS_finit_module+0xe/0x10
[    36.785892]   [<00000000eb46fca8>] entry_SYSCALL_64_fastpath+0x1c/0xbb
[    36.785894] Memory state around the buggy address:
[    36.785898]   00000000507ddd10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[    36.785903]   0000000080e3d449: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[    36.785907] >000000006a8fba16: 00 00 00 00 00 00 00 00 00 00 00 00 00 fc fc fc
[    36.785908]                                                                     ^
[    36.785912]   0000000071be1d0f: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
[    36.785916]   000000004d3d0483: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
[    36.785918] ==================================================================
```

反汇编找到源码，看到是在 mpi_read_raw_from_sgl 函数中对 sg 缓冲区的操作

```
   0xffff80000065c160 <+832>:bl         0xffff8000002fe4a0 <__asan_load1>
   0xffff80000065c164 <+836>:lsl        x19, x19, #8
   0xffff80000065c168 <+840>:tst        x23, #0x7
   0xffff80000065c16c <+844>:sxtw       x22, w21
506             for_each_sg(sgl, sg, ents, i) {
507                     const u8 *buffer = sg_virt(sg) + lzeros;
508                     int len = sg->length - lzeros;
509                     int buf_shift = x;
510
511                     if   (sg_is_last(sg) && (len % BYTES_PER_MPI_LIMB))
512                             len += BYTES_PER_MPI_LIMB - (len % BYTES_PER_MPI_LIMB);
```

```
513
514                 for (; x < len + buf_shift; x++) {
515                         a <<= 8;
516                         a |= *buffer++;//buffer 读越界
517                         if (((z + x + 1) % BYTES_PER_MPI_LIMB) == 0) {
518                                 val->d[j--] = a;
519                                 a = 0;
520                         }
521                 }
522                 z += x;
523                 x = 0;
524                 lzeros = 0;
525         }
```

代码不熟怎么办，打开 linux 开源社区搜到一个一模一样的问题，buffer 的长度计算错误导致访问越界。https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=0bb5c9ead62f4e1eaf226744076e0b4e8348f68f

```
The exact purpose of the len extension isn't clear to me, but due to
its form, I suspect that it's a leftover somehow accounting for leading
zero bytes within the most significant output limb.

Note however that without that len adjustement, the total number of bytes
ever processed by the inner loop equals nbytes and thus, the last output
limb gets written at this point. Thus the net effect of the len adjustement
cited above is just to keep the inner loop running for some more
iterations, namely < BYTES_PER_MPI_LIMB ones, reading some extra bytes from
beyond the last SGE's buffer and discarding them afterwards.

Fix this issue by purging the extension of len beyond the last input SGE's
buffer length.

Fixes: 2d4d1eea540b ("lib/mpi: Add mpi sgl helpers")
Signed-off-by: Nicolai Stange <nicstange@gmail.com>
Signed-off-by: Herbert Xu <herbert@gondor.apana.org.au>

Diffstat
-rw-r--r-- lib/mpi/mpicoder.c 3
1 files changed, 0 insertions, 3 deletions

diff --git a/lib/mpi/mpicoder.c b/lib/mpi/mpicoder.c
index a9f1097c18cb71..747606f9e4a355 100644
--- a/lib/mpi/mpicoder.c
+++ b/lib/mpi/mpicoder.c
@@ -484,9 +484,6 @@ MPI mpi_read_raw_from_sgl(struct scatterlist *sgl, unsigned int nbytes)
                 const u8 *buffer = sg_virt(sg) + lzeros;
                 int len = sg->length - lzeros;

-                if  (sg_is_last(sg) && (len % BYTES_PER_MPI_LIMB))
-                        len += BYTES_PER_MPI_LIMB - (len % BYTES_PER_MPI_LIMB);
-
                 for (x = 0; x < len; x++) {
                         a <<= 8;
                         a |= *buffer++;
```

4.2 声卡驱动有内存 use-after-free

```
[    29.931243] ==================================================================

[    29.940663] BUG: KASAN: use-after-free in snd_hdac_i915_register_notifier+0x24/0x40 [snd_hda_core] at addr 00000000a26014ef

[    29.950494] Write of size 8 by task modprobe/495

[    29.960409] ==================================================================================

[    29.970667] BUG kmalloc-64 (Not tainted): kasan: bad access detected

[    29.978827] ----------------------------------------------------------------------------

......

[    29.999338] CPU: 0 PID: 495 Comm: modprobe Tainted: G      B              4.4.131-20230505.kylin.x86+ #2

[    29.999342] Hardware name: Default string Default string/Default string, BIOS 5.11 09/08/2023

[    29.999349]   ffff8803e0578000 195240857187230a ffff8803db3d74c0 ffffffff81640518

[    29.999354]   ffff8803ec407700 ffff8803e05793e8 ffff8803db3d74f0 ffffffff81309779

[    29.999358]   ffff8803ec407700 ffffea000f815e00 ffff8803e05793e8 0000000000000001

[    29.999359] Call Trace:
```

```
[    29.999366]  [<00000000bfe3e791>] dump_stack+0x63/0x8b
[    29.999372]  [<000000007c500c3c>] print_trailer+0xf9/0x150
[    29.999378]  [<00000000dde5b415>] object_err+0x34/0x40
[    29.999384]  [<00000000b7545eac>] kasan_report.part.2+0x232/0x530
[    29.999416]  [<000000004c52129e>] ? snd_hdac_i915_register_notifier+0x24/0x40 [snd_hda_core]
[    29.999422]  [<00000000065d204b>] ? __mutex_init+0x51/0x60
[    29.999429]  [<00000000093859d4>] kasan_report+0x25/0x30
[    29.999434]  [<00000000a0ced2d4>] __asan_store8+0x60/0x70
[    29.999463]  [<000000004c52129e>] snd_hdac_i915_register_notifier+0x24/0x40 [snd_hda_core]
[    29.999487]  [<00000000cc4255c1>] patch_generic_hdmi+0xbcc/0xd70 [snd_hda_codec_hdmi]
[    29.999513]  [<000000009f914cb4>] ? hdmi_pin_setup_infoframe+0x2e0/0x2e0 [snd_hda_codec_hdmi]
[    29.999519]  [<00000000fbdf6e3f>] ? devres_add+0x9a/0xc0
[    29.999548]  [<00000000121d0df1>] ? hda_reg_write.part.2+0x2d0/0x2d0 [snd_hda_core]
[    29.999554]  [<000000009aa7ac85>] ? __regmap_init+0x13fd/0x14c0
[    29.999559]  [<00000000651a7758>] ? snprintf+0x98/0xd0
[    29.999603]  [<000000003424bf61>] hda_codec_driver_probe+0xc1/0x190 [snd_hda_codec]
[    29.999611]  [<0000000085b8191d>] driver_probe_device+0x2d2/0x6c0
[    29.999619]  [<00000000457e8320>] __driver_attach+0x11a/0x120
[    29.999625]  [<00000000efa7645c>] ? driver_probe_device+0x6c0/0x6c0
[    29.999631]  [<0000000012a91617>] bus_for_each_dev+0xf3/0x170
[    29.999638]  [<0000000049aabc22>] ? subsys_dev_iter_exit+0x10/0x10
[    29.999644]  [<00000000c832d6a7>] ? kasan_kmalloc+0x5e/0x70
[    29.999650]  [<000000003623b2c9>] ? klist_node_init+0x57/0x80
[    29.999658]  [<000000007b691b3d>] driver_attach+0x2b/0x30
```

```
int snd_hdac_i915_register_notifier(const struct i915_audio_component_audio_ops *aops)
{
        if (!hdac_acomp)
                return -ENODEV;

        hdac_acomp->audio_ops = aops;
        return 0;
}
```

在 snd_hdac_i915_register_notifier 中 hdac_acomp 已经在之前被释放，但是没指向 NULL，在这里再次访问的时候就会报错。需要找到 hdac_acomp 是在哪里释放的。

注意在 kasan 报出 bug 之前声卡打印了一行错误信息：

```
[    29.833452] snd_hda_intel 0000:00:1f.3: failed to add i915_bpo component master (-19)
```

查看代码是在 snd_hdac_i915_init_bpo 时候错误处理时候释放了内存但是没把全局指针 hdac_acomp 指向 null，导致有野指针。再参考 snd_hdac_i915_init 函数中正确的用法，在错误处理时候释放内存之后要把指针指向 null。

```
int snd_hdac_i915_init_bpo(struct hdac_bus *bus)
{
        struct component_match *match = NULL;
        struct device *dev = bus->dev;
        struct i915_audio_component *acomp;
        int ret;
```

```
261
262         acomp = kzalloc(sizeof(*acomp), GFP_KERNEL);
263         if (!acomp)
264                 return -ENOMEM;
265         bus->audio_component = acomp;
266         hdac_acomp = acomp;
......
289 out_err:
290         kfree(acomp);//free 之后没指向 null，hdac_acomp 变成野指针
291         bus->audio_component = NULL;
292         dev_err(dev, "failed to add i915_bpo component master (%d)\n", ret);
293
294         return ret;
295 }
```

```
211 int snd_hdac_i915_init(struct hdac_bus *bus)
212 {
213         struct component_match *match = NULL;
214         struct device *dev = bus->dev;
215         struct i915_audio_component *acomp;
216         int ret;
217
218         acomp = kzalloc(sizeof(*acomp), GFP_KERNEL);
219         if (!acomp)
220                 return -ENOMEM;
221         bus->audio_component = acomp;
222         hdac_acomp = acomp;
......
245 out_err:
246         kfree(acomp);
247         bus->audio_component = NULL;
248         hdac_acomp = NULL;
249         dev_info(dev, "failed to add i915 component master (%d)\n", ret);
250
251         return ret;
252 }
```